Software-Defined Networking Firewall

Using Mininet, POX, and OpenFlow

Author: Xinyi Ping

University of California, Santa Cruz

June 2026

1. Introduction

Software-Defined Networking (SDN) separates the control plane from the data plane, allowing network behavior to be programmed through a centralized controller. This SDN-Firewall project implements an OpenFlow-based SDN firewall using POX and Mininet. The controller performs packet forwarding while enforcing security policies to isolate different hosts and subnets.

2. System Design

Device	Mininet Name	IP Address	Description
Floor 1 Hosts	h101, h102, h103, h104	128.114.1.101/24 128.114.1.102/24 128.114.1.103/24 128.114.1.104/24	Computers on floor 1 of the Department A in the company.
Floor 2 Hosts	h201, h202, h203, h204	128.114.2.201/24 128.114.2.202/24 128.114.2.203/24 128.114.2.204/24	Computers on floor 2 of the Department B in the company.
Trusted Host	h_trust	192.47.38.109/24	A trusted computer outside our network. This host is owned by certified employee from Department A.
Untrusted Host	h_untrust	108.35.24.113/24	An untrusted computer outside our network. We treat this computer as a potential hacker.
LLM Server	h_server	128.114.3.178/24	A server used by our internal or trusted hosts.

***Figure 1**. Hosts and their IP addresses.*

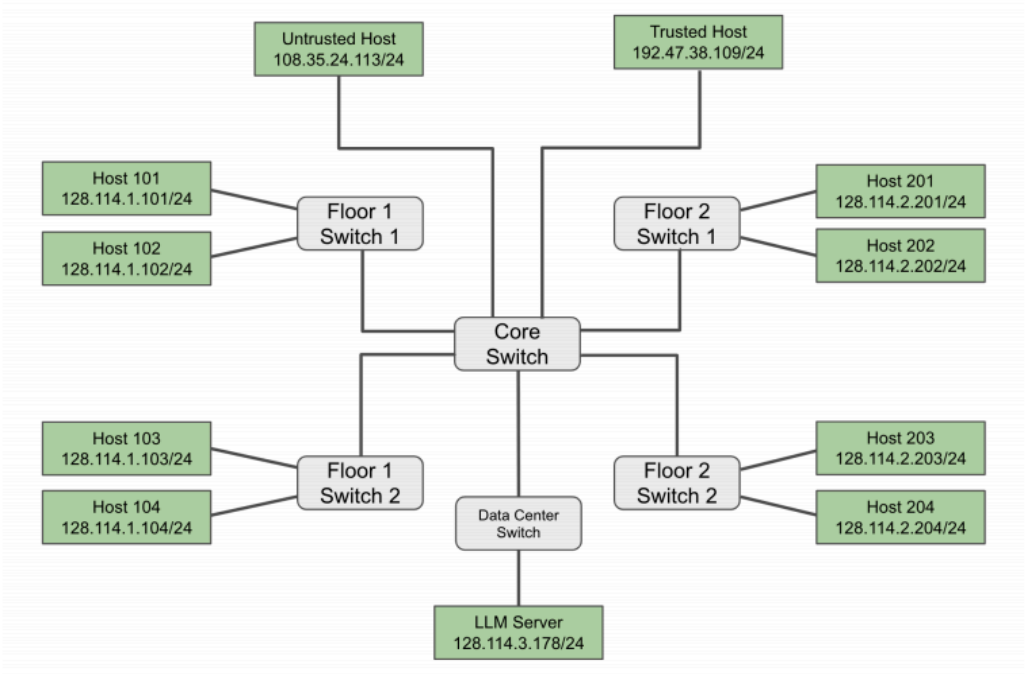

***Figure 2**. Network topology.*

The network consists of six OpenFlow switches and multiple hosts. A POX controller is responsible for installing forwarding rules and enforcing firewall policies.

3. Implementation

The POX controller handles incoming packets through the PacketIn event generated by OpenFlow switches. Whenever a switch receives a packet that does not match any existing flow entry, it forwards the packet to the controller for processing. The controller first parses the received packet and extracts the necessary header information, including the source IP address, destination IP address, switch ID, and ingress port. Based on this information, the controller determines whether the packet should be forwarded or dropped according to the predefined forwarding and firewall policies. Instead of processing every packet individually, the controller installs flow entries into the switch after making a forwarding decision. This allows subsequent packets with the same matching fields to be processed directly by the switch, reducing controller overhead and improving forwarding performance.

The forwarding logic determines the output port for each packet based on the switch where the packet is received and the packet's destination. Unlike a traditional learning switch, this controller does not rely on flooding (OFPP_FLOOD) or normal switching (OFPP_NORMAL). Instead, forwarding rules are explicitly installed using OpenFlow flow entries. Each flow entry matches packets according to their header fields and forwards them to the corresponding output port. This approach provides deterministic routing behavior and allows forwarding decisions to be combined with firewall policies.

Before installing forwarding rules, the controller evaluates each packet against a set of predefined firewall policies. The firewall is implemented by examining the source and destination IP addresses of each packet. If a packet violates one of the security policies, the controller installs a flow entry without any output action, effectively dropping all matching packets.

The implemented firewall policies include:

Source	**Destination**	**Protocol**	**Action**
Untrusted	Internal Hosts	ICMP	Block
Untrusted	LLM Server	Any	Block
Department A	Department B	ICMP	Block
Others			Allow

This centralized policy enforcement demonstrates one of the major advantages of Software-Defined Networking, where security policies can be implemented and updated directly from the controller without modifying individual switches.

Packet forwarding and filtering are implemented using OpenFlow FlowMode messages. For each permitted packet, the controller creates a flow entry that matches the packet header and specifies the corresponding output port. Both idle and hard timeouts are configured so that inactive flow entries are automatically removed after a period of time. By installing flow entries on the switches, subsequent packets matching the same flow are processed locally without generating additional PacketIn events. This mechanism significantly reduces communication between the switches and the controller while maintaining the required forwarding and firewall behavior.

4. Experimental Results

Basic Connectivity

The forwarding rules were first verified by testing connectivity between hosts that were permitted to communicate.

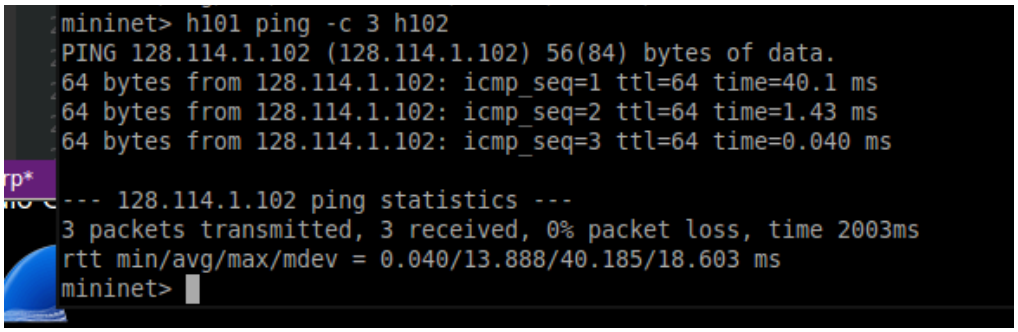

***Figure 3**. ICMP packets from h101 successfully reached h102.*

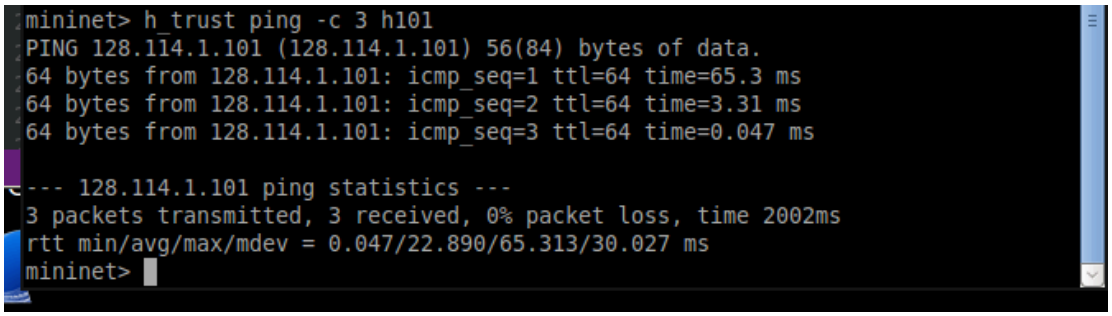

***Figure 4**. ICMP packets from h_trust successfully reached h101.*

Firewall Verification

Firewall policies were validated by testing communication between hosts that should be denied access.

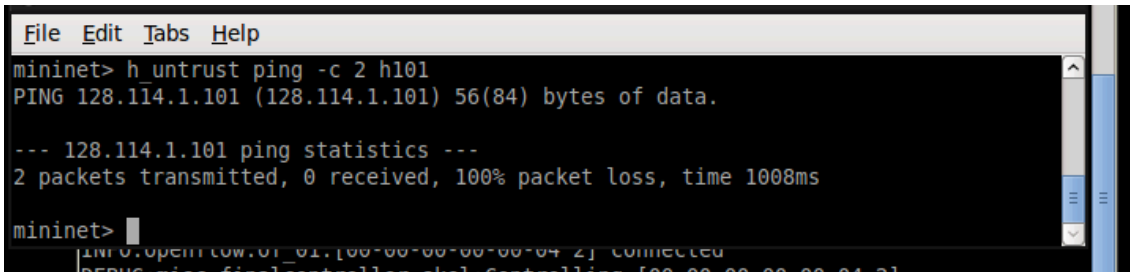

Figure 5*. ICMP requests from the untrusted host to h101 resulted in 100% packet loss.*

Flow Table Inspection

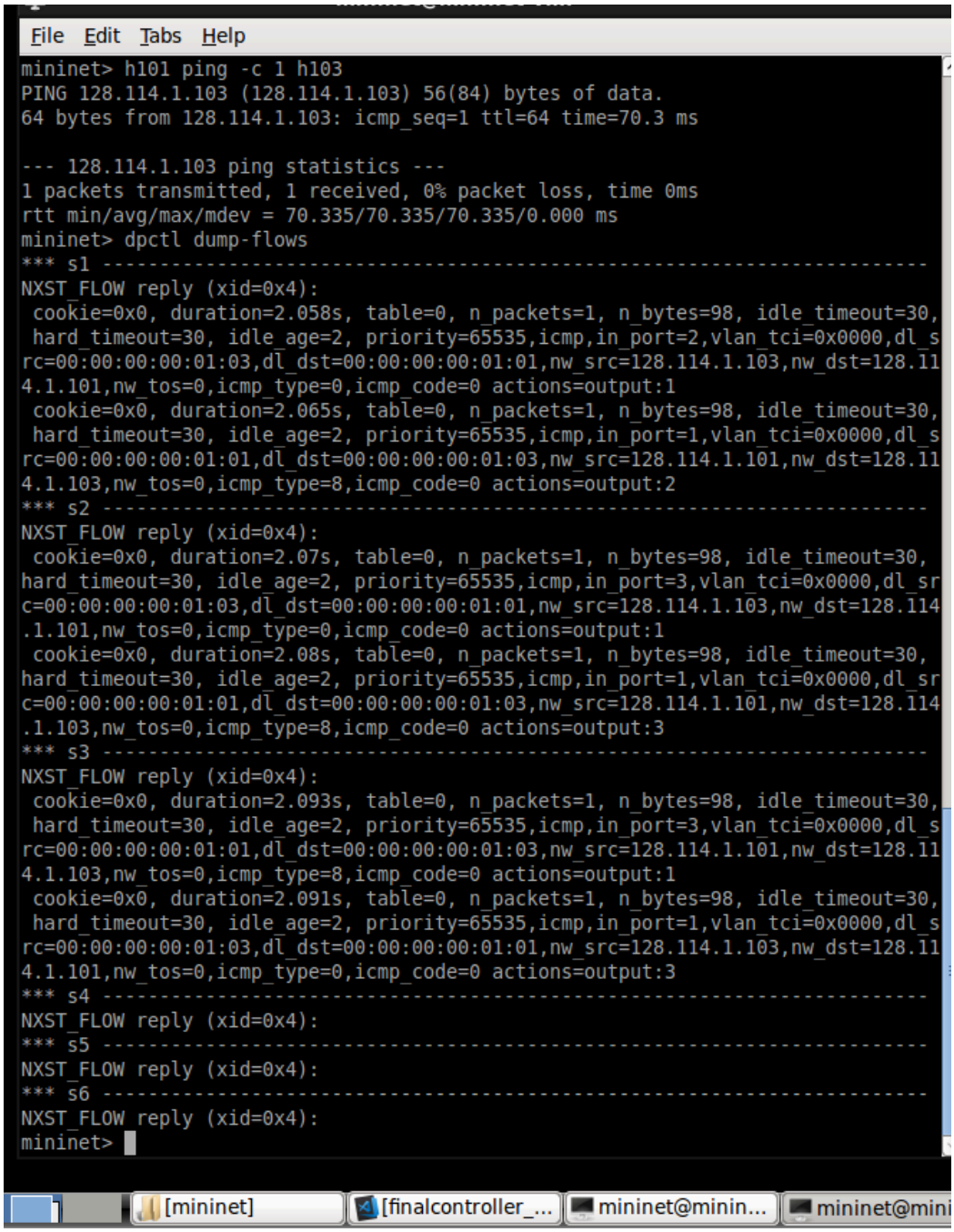

Figure 6. *Flow entries were successfully installed after the first packet was processed by the controller.*

5. Conclusion

This project demonstrates how SDN enables centralized traffic management and security enforcement. By combining Mininet, OpenFlow, and a custom POX controller, packet forwarding and access-control policies were successfully implemented and validated.

The controller successfully performs forwarding without relying on flooding. Security policies are enforced centrally through OpenFlow rules. One limitation is that the forwarding table is statically configured and does not support dynamic routing.